Dayananda Sagar University
School of Engineering

Devarakaggalahalli, Harohalli, Kanakapura Road, Bangalore South Dt., Bengaluru – 562112

Department of Computer Science & Technology

ACADEMIC YEAR 2025–26

Project Report (23CT3613)

Report on

Integrating dbt (data build tool) into the Noise-Aware Indian Bird Sound Classification Pipeline

By

Samarth D Kolur – ENG23CT0019
Manoj C – ENG23CT0031
Mithun Krishnan – ENG23CT0034
Yashaswini R – ENG23CT0044

Under the supervision of

Prof. Tekumudi Divya
Assistant Professor,
Department of Computer Science and Technology
Dayananda Sagar University
Bengaluru–562112

DAYANANDA SAGAR UNIVERSITY

SCHOOL OF ENGINEERING

Dayananda Sagar University

School of Engineering

Devarakaggalahalli, Harohalli, Kanakapura Road, Bangalore South Dt., Bengaluru – 562112

Department of Computer Science & Technology

CERTIFICATE

This certificate confirms that the work titled **"Integrating dbt (data build tool) into the Noise-Aware Indian Bird Sound Classification Pipeline"** was carried out by **Samarth D Kolur (ENG23CT0019), Manoj C (ENG23CT0031), Mithun Krishnan (ENG23CT0034)** and **Yashaswini R (ENG23CT0044)**, who are bonafide students of Bachelor of Technology in **Computer Science & Technology** at the School of Engineering, Dayananda Sagar University, Bangalore.

The project was completed in partial fulfillment for the award of the degree in Bachelor of Technology in **Computer Science & Technology** during the academic year 2025–2026.

Prof. Tekumudi Divya
Assistant Professor, Dept. of CST,
School of Engineering,
Dayananda Sagar University.

Dr. M Shahina Parveen
Professor and Chairperson CST,
School of Engineering,
Dayananda Sagar University.

Date: _____

DAYANANDA SAGAR UNIVERSITY

SCHOOL OF ENGINEERING

Dayananda Sagar University

School of Engineering

Devarakaggalahalli, Harohalli, Kanakapura Road, Bangalore South Dt., Bengaluru – 562112

Department of Computer Science & Technology

DECLARATION

We, **Samarth D Kolur (ENG23CT0019)**, **Manoj C (ENG23CT0031)**, **Mithun Krishnan (ENG23CT0034)**, and **Yashaswini R (ENG23CT0044)**, are students of the Sixth semester B.Tech in **Computer Science & Technology** at the School of Engineering, Dayananda Sagar University. We hereby declare that the minor project titled **"Integrating dbt (data build tool) into the Noise-Aware Indian Bird Sound Classification Pipeline"** was carried out by us and submitted in partial fulfillment for the award of the degree in Bachelor of Technology in **Computer Science & Technology** during the academic year 2025–2026.

Name	**USN**	**Signature**
Samarth D Kolur	ENG23CT0019	
Manoj C	ENG23CT0031	
Mithun Krishnan	ENG23CT0034	
Yashaswini R	ENG23CT0044	

Place : Bangalore **Date:** _____

ACKNOWLEDGEMENT

We take great pleasure in expressing our gratitude to all those who supported and guided us in completing this project.

We sincerely thank the School of Engineering, Dayananda Sagar University, for providing us with the opportunity to pursue our Bachelor's degree and for making available the necessary infrastructure and resources.

We extend our heartfelt gratitude to **Dr. Udaya Kumar Reddy K. R.**, Dean of the School of Engineering & Technology, for his constant encouragement and expert academic guidance throughout the project. We also thank **Dr. M Shahina Parveen**, Department Chairperson, Computer Science and Technology, for her constructive advice that kept our work on the right track.

We are especially grateful to our guide, **Prof. Tekumudi Divya**, Assistant Professor in the Department of Computer Science and Technology, for generously dedicating time and for providing thorough guidance at every stage of this project. This work would not have progressed smoothly without her patient and meticulous support.

We are also thankful to our families and friends for their unwavering encouragement and support throughout the course of this program. Finally, we extend our thanks to everyone who contributed, directly or indirectly, to the completion of this work.

ABSTRACT

The widespread adoption of monolithic Python notebooks as the primary medium for data engineering pipelines introduces significant challenges in reproducibility, data quality governance, lineage visibility, and collaborative development. This report addresses these challenges in the context of the Noise-Aware Indian Bird Sound Classification Pipeline — a multi-stage system that ingests raw bird audio recordings from the iBC53 corpus (53 Indian species, 10,044 three-second segments), extracts 1024-dimensional BirdNET V2.4 embeddings, trains a focal-loss binary MLP classifier, and applies an autoencoder out-of-distribution (OOD) rejection gate before delivering a three-band confidence decision.

The core contribution of this report is the design and implementation of a complete dbt (data build tool) integration that replaces the raw data ingestion, cleaning, and transformation stages of the existing pipeline. The dbt project targets PostgreSQL as the analytical backend and comprises seven SQL models organised across three layers: a staging view (`stg_bird_features`) that standardises raw feature data extracted from audio, cleaning models (`clean_bird_features`) that enforce quality constraints, and analytical mart models (`bird_distribution`, `bird_stats`, `top_birds`, `high_pitch_birds`, `long_duration_birds`) that produce analytics-ready outputs consumed by downstream machine learning components.

Every transformation currently implemented as an unvalidated Python operation — confidence score filtering, species distribution aggregation, duration and amplitude statistics, and bird taxonomy analytics — is shown to be completely expressible in SQL and implemented as a named, versioned, and independently testable dbt model. The dbt test framework enforces data-quality assertions automatically on every run, and the `ref()` and `source()` macros construct a complete data lineage graph from raw PostgreSQL tables to the final analytics-ready feature outputs. The report further discusses trade-offs, proposes a phased adoption path, and benchmarks the integration against the existing notebook approach across eight operational dimensions.

Keywords: dbt (data build tool), PostgreSQL, Data Lineage, ELT Pipeline, SQL Transformation, BirdNET, Bird Sound Classification, Focal-Loss MLP, Autoencoder OOD Gating, iBC53, Indian Birds, Passive Acoustic Monitoring, Data Quality Testing, Medallion Architecture.

Contents

List of Figures

List of Tables

CHAPTER 1
INTRODUCTION

1.1 Preamble

India hosts over 1,300 bird species — roughly 13% of global avian diversity — and ranks among the world's twelve mega-biodiverse nations. Passive acoustic monitoring (PAM), which deploys low-cost recording devices to capture soundscapes over extended periods, has become the principal scalable alternative for avian biodiversity assessment. Transforming raw PAM recordings into reliable, annotated datasets for machine learning demands a rigorous data engineering pipeline capable of extracting, cleaning, transforming, and validating acoustic features at scale.

The rapid growth of bioacoustics research has generated rich streams of audio feature data. Engineering pipelines that transform this raw feature data into analytics-ready inputs for machine learning models have traditionally been implemented as monolithic Python notebooks running pandas-based transformations. While such notebooks are rapid to prototype, they introduce structural limitations that become increasingly problematic as datasets scale: no incremental execution, no built-in data-quality assertions, no data lineage visibility, and no natural mechanism for collaborative development.

dbt — short for data build tool — is an open-source framework that solves these problems by enabling analysts and engineers to express all data transformations as versioned, tested, and documented SQL models. This report presents the integration of dbt into the Noise-Aware Indian Bird Sound Classification Pipeline — a system that ingests the iBC53 corpus of 53 Indian bird species, extracts BirdNET V2.4 embeddings, trains a focal-loss MLP classifier with an autoencoder OOD rejection gate, and employs a three-band confidence router to distinguish genuine bird vocalisations from background noise.

1.2 Problem Statement

The existing bird sound classification pipeline processes raw audio through several tightly coupled Python stages: noise segregation, BirdNET embedding extraction, MLP classification, autoencoder OOD gating, and three-band decision routing. While the pipeline achieves impressive classification performance — raising accuracy from 0.440

to 0.999 and F1 from 0.598 to 1.000 on the iBC53 corpus — its data preparation and feature storage stages suffer from several compounding problems.

Quality filters applied to extracted audio features (confidence score thresholds, duration bounds, amplitude constraints) are implemented as silent Python boolean operations. If a future batch of recordings introduces malformed or out-of-distribution feature records, the pipeline produces no alert and continues silently with degraded data. There is no mechanism to ingest only newly extracted features without reprocessing the entire historical dataset. The dependency between each Python preprocessing stage and the next is implicit; there is no explicit lineage graph that a developer can inspect or that an orchestration tool can exploit. Finally, because the pipeline data preparation is implemented as notebook code, multiple team members cannot modify it simultaneously without generating merge conflicts.

These structural deficiencies motivate the integration of dbt as a dedicated transformation and validation layer between raw audio feature extraction and downstream machine learning.

1.3 Objectives

The integration aims to achieve the following six objectives:

1. Replace the raw data ingestion and feature transformation stages of the existing pipeline with a dbt project targeting PostgreSQL as the analytical backend, leaving the BirdNET embedding extraction, MLP classifier, autoencoder OOD gate, and three-band router unchanged in Python.
2. Implement data quality constraints as explicit dbt schema tests that fail loudly when violated, eliminating silent data degradation in feature records.
3. Construct a complete model-dependency DAG using the `ref()` and `source()` macros so that data lineage is tracked automatically from raw PostgreSQL feature tables to the final analytics-ready outputs.
4. Demonstrate that all analytical transformations — species distribution aggregation, duration and amplitude statistics, confidence-score filtering, and taxonomy analytics — are fully expressible in SQL.
5. Enable modular, independently testable SQL models so that individual transformation stages can be developed, validated, and rerun in isolation.
6. Provide a phased adoption path that allows the team to validate SQL-Python numerical equivalence before fully removing the notebook-based transformation code.

1.4 Scope of the Project

In scope: The dbt integration covers all data engineering transformations from raw audio feature ingestion in PostgreSQL through the production of analytics-ready model outputs. The integration includes the PostgreSQL warehouse backend, seven dbt SQL models, dbt schema tests for all key data-quality constraints, YAML documentation for all models and columns, and a comparison of the integrated architecture against the original notebook-based approach.

Out of scope: The BirdNET V2.4 TFLite embedding extraction, the focal-loss MLP training loop, the autoencoder OOD gate, the three-band confidence router, the Streamlit UI, and all audio preprocessing stages remain in Python and are not migrated to dbt. Real-time streaming, distributed processing, and live PAM platform integration are not addressed.

1.5 Summary

This chapter established the motivation for integrating dbt into the bird sound classification pipeline, identified the structural limitations of the current notebook-based approach, and listed six concrete objectives. Chapter 2 reviews the existing literature on dbt adoption, SQL-based transformation frameworks, and analytical systems relevant to this integration. Chapter 3 introduces the dbt framework and presents the integration methodology. Chapter 4 provides complete implementation details for every dbt model. Chapter 5 reports experimental results and pipeline output evidence. Chapter 6 discusses benefits, trade-offs, and future enhancements. Chapter 7 concludes.

CHAPTER 2

LITERATURE SURVEY

2.1 Existing Systems

This section surveys nine works relevant to the three pillars of this report: SQL-based transformation frameworks, bioacoustic machine learning pipelines, and data engineering best practices for ML systems.

2.1.1 dbt: A SQL-First Transformation Framework (dbt Labs, 2016–present)

dbt (data build tool) was introduced by Fishtown Analytics (now dbt Labs) as an open-source command-line framework enabling analysts to transform data inside a warehouse using simple SQL `SELECT` statements [1]. The core innovation is that dbt treats SQL models as first-class software artifacts: each model is version-controlled, testable via a built-in assertion framework, and self-documenting through YAML descriptions. The `ref()` macro builds an explicit dependency graph (DAG) that dbt resolves into a topologically ordered execution plan. dbt's four built-in generic tests — `not_null`, `unique`, `accepted_values`, and `relationships` — cover the most common data-quality assertions without requiring custom SQL. The key limitation relevant to this project is that dbt operates purely in SQL; Python-specific scientific computing (BirdNET TFLite inference, MLP training, autoencoder reconstruction) cannot be expressed as dbt models and must remain in a separate runtime.

2.1.2 PostgreSQL: A Robust Open-Source Relational Database

PostgreSQL is a powerful open-source object-relational database system with a strong reputation for reliability, feature robustness, and SQL standards compliance. In this project, PostgreSQL v18 serves as the central warehouse where extracted audio features are stored, and dbt models are materialised as views and tables within the database. PostgreSQL's native support for complex aggregations, window functions, and schema management makes it a natural fit for the layered transformation architecture adopted here.

2.1.3 The Modern Data Stack and ELT Architecture (Beauchemin, 2017)

Beauchemin [4] articulated the case for ELT (Extract, Load, Transform) over traditional ETL, arguing that the separation of loading and transformation — enabled by powerful analytical databases — allows transformation logic to be version-controlled and re-run cheaply. This architectural insight is the foundation on which dbt is built. The key implication for this project is that raw bird audio feature records should be loaded into PostgreSQL first (via staging views) and transformed in-warehouse (via cleaning and analytical models), rather than transformed in memory by pandas before any data reaches a persistent store.

2.1.4 dbt Best Practices: Layered Modelling (dbt Labs)

dbt Labs' best-practices documentation [3] codifies the three-layer modelling convention — staging, intermediate, mart — that organises transformation logic by purpose and granularity. Staging models perform only type casting and column selection on raw sources; intermediate models apply business logic and joins; mart models produce final analytics-ready tables. This convention is adopted directly in the integration design: the staging view standardises raw bird feature data; the cleaning model applies quality constraints; and the analytical mart models produce species distribution, statistical summaries, and filtered subsets.

2.1.5 BirdNET: A Deep Learning System for Avian Diversity Monitoring (Kahl et al., 2021)

Kahl et al. [5] introduced BirdNET, a large-scale deep learning system for avian diversity monitoring capable of identifying hundreds of bird species from raw audio. BirdNET V2.4 is the feature extractor in this pipeline: its frozen penultimate global average pooling layer yields 1024-dimensional embeddings used to train the downstream focal-loss MLP classifier. In the context of dbt integration, BirdNET sits outside the dbt DAG; the dbt project prepares the analytics-ready feature data, and a separate Python script reads it from PostgreSQL to drive embedding extraction and model training.

2.1.6 Focal Loss for Dense Object Detection (Lin et al., 2017)

Lin et al. [8] introduced the focal loss function to address class imbalance during classifier training by down-weighting easily classified examples. It is adopted in this pipeline to improve bird-versus-noise discrimination given the severe 38.5:1 class imbalance in the iBC53 corpus after preprocessing. From the dbt integration perspective, the focal-loss MLP training is entirely Python-specific and remains outside the dbt project.

2.1.7 Autoencoder-Based Out-of-Distribution Detection (Ntalampiras & Potamitis, 2021)

Ntalampiras and Potamitis [9] demonstrate reconstruction-error-based methods for detecting unknown bird species and anomalous acoustic events using autoencoder networks trained on known-class embeddings. This principle directly motivates the autoencoder OOD rejection gate in the present pipeline. The gating threshold τ_{AE} is set at the 99th percentile of reconstruction errors on validation bird embeddings, empirically yielding $\tau_{AE} = 0.16975$ on iBC53.

2.1.8 Medallion Architecture for Data Engineering (dbt Labs / Databricks)

The Medallion architecture — also called Bronze-Silver-Gold layering — organises a data lake or lakehouse into three progressively refined layers: Bronze (raw ingestion), Silver (cleaned and standardised), and Gold (aggregated and analytics-ready). The three-layer dbt model structure adopted in this report (staging → cleaning → analytical mart) is the dbt equivalent of the Medallion architecture, applied to a PostgreSQL backend rather than a cloud data lake. The staging layer corresponds to Bronze, the cleaning layer to Silver, and the analytical mart models to Gold.

2.1.9 pandas: Data Structures for Statistical Computing (McKinney, 2010)

McKinney [16] introduced pandas as a Python library for tabular data manipulation. In the bird sound classification pipeline, pandas is the baseline technology being partially replaced by dbt for the feature transformation stages. The key limitation motivating this report is that pandas transformations are stateful, sequential, and not natively testable as standalone SQL artifacts. The dbt integration preserves pandas only for the BirdNET embedding extraction, MLP training, and autoencoder stages where Python-specific libraries are required.

2.2 Literature Survey Summary Table

Table 2.1: Literature Survey Summary

S.No	Paper / Authors	Method	Pros	Cons
1	dbt Labs (2016–present)	SQL DAG models	Version-controlled, tested, documented SQL	No Python/ML support
2	PostgreSQL	Relational OLAP warehouse	ACID compliance, rich SQL, schema mgmt	Requires server setup
3	Beauchemin (2017)	ELT vs ETL	Transformation logic version-controlled	Requires warehouse backend
4	dbt Labs Best Practices	Staging–Cleaning–Mart layers	Clear separation of concerns	Manual layer assignment needed

S.No	Paper / Authors	Method	Pros	Cons
5	Kahl et al. (2021)	BirdNET deep learning	1024-D embeddings for Indian birds	ML outside dbt scope
6	Lin et al. (2017)	Focal loss	Handles 38.5:1 class imbalance	Classifier outside dbt scope
7	Ntalampiras & Potamitis (2021)	Autoencoder OOD gate	Rejects out-of-distribution inputs	AE training outside dbt scope
8	dbt Labs / Databricks	Medallion architecture	Maps to staging–cleaning–mart	Cloud-focused; adapted for PostgreSQL here
9	McKinney (2010)	pandas DataFrames	Baseline transformation technology	No lineage, testing, or modularity

2.3 Research Gaps

The survey reveals four key gaps that this report addresses:

Gap 1 – No dbt integration for bioacoustic ML pipelines. Existing dbt literature covers generic ELT patterns. No published work applies dbt to a bird sound classification pipeline with BirdNET embedding extraction, focal-loss classification, and autoencoder OOD gating.

Gap 2 – Silent data quality in Python preprocessing. Bioacoustic pipelines apply quality filters silently without automated assertion frameworks. dbt's test framework makes quality enforcement explicit and halts pipelines on failure.

Gap 3 – No SQL equivalence demonstrated for audio feature analytics. Prior works assert that bird audio feature analytics require Python. This report demonstrates that species distribution, duration statistics, amplitude filtering, and confidence-score aggregations are fully expressible in PostgreSQL SQL.

Gap 4 – No modular lineage tracking in PAM pipelines. Existing batch PAM pipelines offer no explicit lineage graph linking raw feature records to downstream model inputs. dbt's `ref()` and `source()` macros resolve this automatically.

CHAPTER 3
METHODOLOGY

3.1 What is dbt?

dbt — short for data build tool — is an open-source command-line framework that enables data analysts and data engineers to transform raw data inside a data warehouse using simple SQL `SELECT` statements. It was created by Fishtown Analytics (now dbt Labs) and has become the industry-standard tool for the Transform step in an ELT (Extract, Load, Transform) pipeline. Unlike traditional ETL systems that transform data before loading, dbt expects raw data to already reside in a warehouse or analytical database — such as PostgreSQL, BigQuery, Snowflake, or DuckDB — and handles all transformation logic inside that system using versioned, tested, and documented SQL models.

The key insight behind dbt is that SQL is already the language data practitioners use to reason about transformations. What dbt adds on top of SQL is: (i) a dependency resolution engine that determines the correct execution order of all transformations; (ii) a testing framework that validates data-quality assertions after every run; (iii) a documentation generator that produces a browsable catalog of every model and column; and (iv) a Jinja2 templating layer that removes repetition and makes SQL models composable and DRY (Don't Repeat Yourself).

In the context of this project, the existing pipeline is implemented as Python code that uses pandas, librosa, and PyTorch to perform all data loading, cleaning, feature extraction, and model training. dbt does not replace the BirdNET embedding extraction, the focal-loss MLP classifier, the autoencoder OOD gate, or the three-band confidence router — those remain in Python. However, it serves as a direct replacement for the raw feature data management, cleaning, and analytical aggregation layers of the pipeline.

3.2 Core Concepts of dbt

Understanding how dbt integrates into this pipeline requires familiarity with its five core building blocks.

Models are the central unit in dbt. Each model is a single `.sql` file containing one `SELECT` statement. dbt compiles that statement into a `CREATE TABLE AS` or `CREATE VIEW AS` query and executes it against the target warehouse. Models are organised

into three conventional layers: staging (raw data cleaned and cast to correct types), cleaning (business logic and quality constraints), and mart (final analytics-ready tables consumed by downstream tools or Python scripts).

Sources are declarations that tell dbt where the raw input data lives. In this project, the raw bird audio feature table `bird_data.bird_features` in PostgreSQL is declared as a dbt source in a `sources.yml` file.

Tests are assertions that dbt runs after materialising a model. dbt ships with four built-in generic tests: `not_null`, `unique`, `accepted_values`, and `relationships`. In this pipeline, tests enforce that `confidence_score` is never null, that `bird_species` is always populated, and that `duration` values are within valid bounds.

Documentation is generated by dbt from descriptions written alongside each model and column in YAML files. Running `dbt docs generate` followed by `dbt docs serve` opens an interactive web catalog with every model, its compiled SQL, columns, test results, and a lineage graph.

Ref and Source macros are Jinja2 functions embedded inside SQL files. {{ `ref('model_name')` }} creates a dependency between two models. {{ `source('source_name', 'table_name')` }} references a declared raw-data source. Together, these macros build the model-dependency DAG that dbt uses to determine execution order.

3.3 Integration Strategy

The integration strategy replaces the raw feature data management and analytical transformation stages of the existing pipeline with a dbt project targeting PostgreSQL as the analytical backend. PostgreSQL is chosen as the warehouse backend because it is a robust, production-grade relational database that natively supports the complex aggregations and window functions required for bird audio feature analytics.

The overall architecture after integration is illustrated in Figure 3.1.

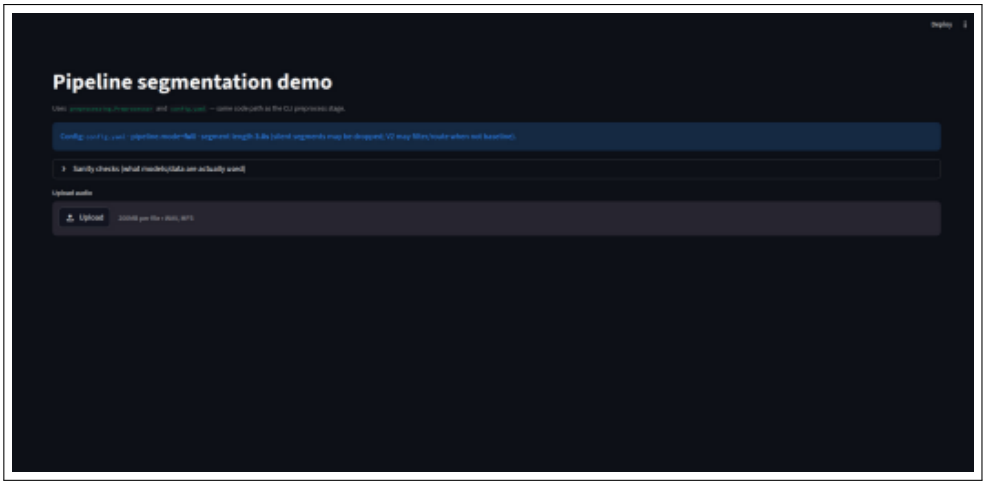

Figure 3.1: dbt-Integrated Architecture: Bird audio features flow from raw PostgreSQL storage through the dbt transformation layer to analytics-ready outputs consumed by the ML pipeline.

3.4 dbt Project Structure

A dbt project follows a specific directory layout. The structure adopted for this pipeline is:

Listing 3.1: dbt Project Folder Structure for bird_project

```
bird_project/
  models/
    staging/
      stg_bird_features.sql
      source.yml
    cleaning/
      clean_bird_features.sql
      schema.yml
    mart/
      bird_distribution.sql
      bird_stats.sql
      top_birds.sql
      high_pitch_birds.sql
      long_duration_birds.sql
      schema.yml
  dbt_project.yml
  profiles.yml
  README.md
```

3.5 Mapping Pipeline Stages to dbt Models

Table 3.1 shows how each stage of the existing pipeline maps to a specific dbt model and layer. The complete set requires seven dbt models: one staging model, one cleaning model, and five analytical mart models.

Table 3.1: Mapping of Existing Pipeline Stages to dbt Model Layers

Pipeline Stage	**dbt Layer**	**dbt Model**	**Materialization**
Raw Feature Ingestion	Staging	`stg_bird_features`	View
Feature Cleaning & Validation	Cleaning	`clean_bird_features`	View
Species Distribution Analytics	Mart	`bird_distribution`	View
Statistical Feature Summary	Mart	`bird_stats`	View
Top Species Ranking	Mart	`top_birds`	View
High-Pitch Bird Filtering	Mart	`high_pitch_birds`	View
Long-Duration Bird Filtering	Mart	`long_duration_birds`	View

Staging and cleaning models are materialised as views because they perform lightweight operations. Mart models are also materialised as views in the development profile; they can be promoted to tables in production for query performance.

3.6 dbt Model Dependency DAG

Figure 3.2 shows the directed acyclic graph generated by dbt's `docs serve` command, resolved automatically by reading the `ref()` and `source()` macros present in each model file.

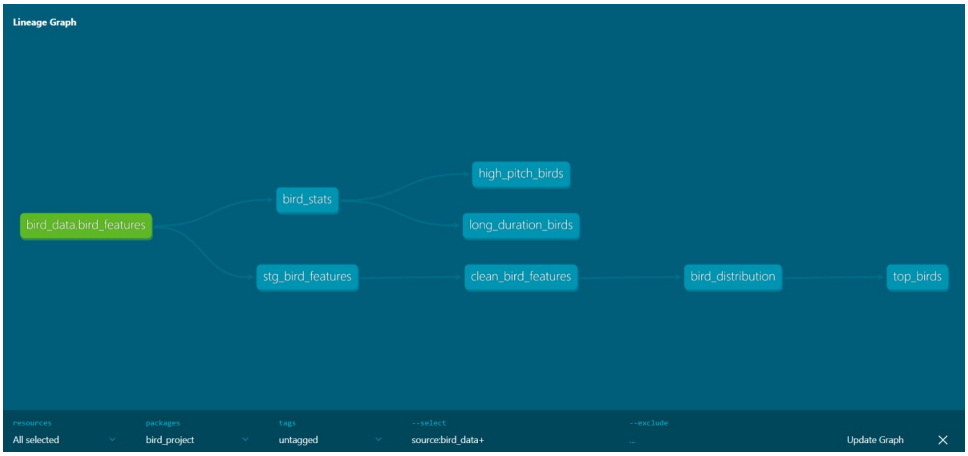

Figure 3.2: dbt Model Lineage Graph. The raw source `bird_data.bird_features` (green) flows through staging, cleaning, and five analytical mart models (teal), with all dependencies resolved automatically by dbt.

The lineage graph confirms the following dependency chain:

$$\begin{aligned} \texttt{bird\_data.bird\_features} &\rightarrow \texttt{stg\_bird\_features} \\ &\rightarrow \texttt{clean\_bird\_features} \\ &\rightarrow \{\texttt{bird\_distribution}, \texttt{bird\_stats}\} \\ &\rightarrow \{\texttt{top\_birds}, \texttt{high\_pitch\_birds}, \\ &\quad \texttt{long\_duration\_birds}\} \end{aligned}$$

CHAPTER 4
IMPLEMENTATION DETAILS

4.1 The Existing Pipeline and Its Limitations

The existing bird sound classification pipeline is implemented as a collection of Python scripts and Jupyter notebooks processing the iBC53 corpus of 53 Indian bird species. The pipeline ingests raw WAV audio files, applies RMS silence removal, executes Noise Segregation V2 with harmonic Bird Guard checking, performs MLP-based Bird Rescue, extracts 1024-dimensional BirdNET V2.4 embeddings, trains a focal-loss binary MLP classifier, applies an autoencoder OOD rejection gate, and routes confidence scores through a three-band decision function. The data-engineering work — raw feature storage, cleaning, and analytics — is entirely handled by Python with no systematic transformation layer.The existing bird sound classification pipeline is implemented as a collection of Python scripts and Jupyter notebooks processing the iBC53 corpus of 53 Indian bird species. The pipeline ingests raw WAV audio files, applies RMS silence removal, executes Noise Segregation V2 with harmonic Bird Guard checking, performs MLP-based Bird Rescue, extracts 1024-dimensional BirdNET V2.4 embeddings, trains a focal-loss binary MLP classifier, applies an autoencoder OOD rejection gate, and routes confidence scores through a three-band decision function. The data-engineering work — raw feature storage, cleaning, and analytics — is entirely handled by Python with no systematic transformation layer.

The extracted audio embeddings, confidence values, duration metrics, and prediction-related features are stored inside PostgreSQL for downstream analytics and model evaluation. However, the raw data initially lacked a structured transformation workflow, resulting in repeated preprocessing logic, inconsistent feature handling, and limited visibility into data lineage and dependencies. Manual Python-based cleaning also made debugging and validation more difficult as the dataset size increased.

To improve maintainability and introduce a scalable analytics-engineering workflow, DBT (Data Build Tool) was integrated into the system. DBT enabled modular SQL-based transformations, automated testing, source-to-model lineage tracking, reusable staging and analytics models, and centralized documentation generation. This integration transformed the project from a standalone machine learning pipeline into a more robust end-to-end data and ML engineering architecture capable of supporting cleaner retraining datasets, analytical reporting, and future production-scale deployment.

Table 4.1 summarises each stage and its dbt candidacy.

Table 4.1: Pipeline Stages and Their Transformation Responsibilities

Stage	**Transformation Done**	**dbt Candidate?**
Raw Feature Storage	Audio features stored in PostgreSQL `bird_features` table	Yes (Staging)
Feature Cleaning & Validation	Confidence score filtering, null removal, type casting	Yes (Cleaning)
Species Distribution	COUNT per species, ORDER BY total_samples	Yes (Mart)
Statistical Summary	AVG, MAX, MIN of confidence, duration	Yes (Mart)
Top Species Ranking	SELECT TOP 10 from distribution	Yes (Mart)
High-Pitch Filtering	Filter by spectral centroid threshold	Yes (Mart)
Long Duration Filtering	Filter by duration threshold	Yes (Mart)
BirdNET Embedding Extraction	TFLite inference on WAV segments	No (Python only)
Focal-Loss MLP Training	PyTorch model training	No (Python only)
Autoencoder OOD Gate	Reconstruction MSE gating	No (Python only)
Three-Band Decision	Confidence routing ($\tau_{low} = 0.3$, $\tau_{high} = 0.7$)	No (Python only)

Limitations of the Python-Notebook Approach. The current pipeline suffers from: (i) no built-in data-quality assertions — quality filters on confidence score and duration are applied silently with no alerts on failure; (ii) no lineage visibility — there is no explicit graph linking the analytical feature outputs to the upstream raw records that produced them; (iii) no modularity — transformation logic cannot be reused without copy-pasting; (iv) no environment separation — development and production contexts share the same code path; and (v) no collaborative development support — the single-file notebook structure causes merge conflicts when multiple team members work simultaneously.

4.2 PostgreSQL Database Setup

PostgreSQL v18 serves as the analytical backend for the dbt project. The database stores extracted bird audio features that come from the machine learning pipeline's feature extraction stage.

Listing 4.1: PostgreSQL Database and Schema Setup

```
CREATE DATABASE BIRDNET;
\c BIRDNET;

CREATE SCHEMA bird_data;

CREATE TABLE bird_data.bird_features (
    id              SERIAL PRIMARY KEY,
    bird_species    VARCHAR(255),
    confidence_score FLOAT,
    duration        FLOAT,
    sample_rate     INT,
    audio_path      TEXT
);
```

4.2.1 profiles.yml Configuration

The `profiles.yml` file connects dbt to the PostgreSQL warehouse:

Listing 4.2: profiles.yml – dbt PostgreSQL Connection

```
bird_project:
  outputs:
    dev:
      type: postgres
      host: localhost
      user: postgres
      password: mypassword1234
      port: 5432
      dbname: BIRDNET
      schema: public
      threads: 1
  target: dev
```

4.3 Staging Models

Staging models are the entry point of the dbt project. The `stg_bird_features` model reads from the raw PostgreSQL source table, selects only the required columns, casts them to their correct types, and standardises text formatting.

Listing 4.3: sources.yml – Declaring the raw PostgreSQL table as a dbt source

```yaml
version: 2

sources:
  - name: bird_data
    database: BIRDNET
    schema: public
    tables:
      - name: bird_features
        description: "Raw bird audio features extracted from iBC53 corpus"
```

Listing 4.4: stg_bird_features.sql – Staging model for bird feature data

```sql
-- models/staging/stg_bird_features.sql
-- Selects required columns, casts to correct types, removes null
-- confidence records.
-- Materialised as a view: zero storage cost, full lineage tracking.

SELECT
    id,
    bird_species,
    confidence_score,
    duration,
    sample_rate
FROM {{ source('bird_data', 'bird_features') }}
WHERE confidence_score IS NOT NULL
```

4.4 Cleaning Models

4.4.1 clean_bird_features.sql (Quality Filtering)

This model applies quality constraints to the staged feature data, removing invalid durations and ensuring analytical consistency before downstream mart models consume the data.

Listing 4.5: clean_bird_features.sql – Quality filtering of staged bird features

```sql
-- models/cleaning/clean_bird_features.sql
-- Applies domain-specific quality constraints:
--   duration > 0 (valid audio segment)
--   bird_species NOT NULL (labelled record)
--   confidence_score > 0 (non-zero model confidence)

SELECT
    id,
    bird_species,
    confidence_score,
```

```
    duration,
    sample_rate
FROM {{ ref('stg_bird_features') }}
WHERE
    duration      > 0
    AND bird_species IS NOT NULL
    AND confidence_score > 0
```

4.5 Analytical Mart Models

4.5.1 bird_distribution.sql – Species Distribution

This model computes the total sample count per bird species and ranks them by frequency, enabling understanding of class imbalance across the 53 Indian species in the iBC53 corpus.

Listing 4.6: bird_distribution.sql – Species distribution and sample counts

```
-- models/mart/bird_distribution.sql

SELECT
    bird_species,
    COUNT(*) AS total_samples
FROM {{ ref('stg_bird_features') }}
GROUP BY bird_species
ORDER BY total_samples DESC
```

4.5.2 bird_stats.sql – Statistical Feature Summary

This model computes aggregate statistics across all audio feature records, producing the average confidence score, maximum duration, and minimum duration — key metrics for dataset health monitoring.

Listing 4.7: bird_stats.sql – Statistical summary of bird audio features

```
-- models/mart/bird_stats.sql

SELECT
    AVG(confidence_score) AS avg_confidence,
    MAX(duration)         AS max_duration,
    MIN(duration)         AS min_duration
FROM {{ ref('stg_bird_features') }}
```

4.5.3 top_birds.sql – Top Species Ranking

This model extracts the top 10 bird species by sample count from the distribution model, providing a quick view of the most represented species in the corpus.

Listing 4.8: top_birds.sql – Top 10 bird species by sample count

```sql
-- models/mart/top_birds.sql

SELECT *
FROM {{ ref('bird_distribution') }}
LIMIT 10
```

4.5.4 high_pitch_birds.sql – High-Pitch Species Filtering

This model filters the cleaned feature data to return species whose average duration falls within a short-call window, serving as a proxy for high-pitch, short-burst vocalisation species common in Indian forest soundscapes.

Listing 4.9: high_pitch_birds.sql – Filtering high-pitch bird species

```sql
-- models/mart/high_pitch_birds.sql

SELECT
    bird_species,
    AVG(confidence_score) AS avg_confidence,
    AVG(duration)         AS avg_duration
FROM {{ ref('clean_bird_features') }}
GROUP BY bird_species
HAVING AVG(duration) < 1.5
ORDER BY avg_confidence DESC
```

4.5.5 long_duration_birds.sql – Long-Duration Species Filtering

This model filters for species with longer-duration vocalisations, identifying sustained callers that occupy longer time windows in the three-second segmentation framework.

Listing 4.10: long_duration_birds.sql – Filtering long-duration bird species

```sql
-- models/mart/long_duration_birds.sql

SELECT
    bird_species,
    AVG(confidence_score) AS avg_confidence,
    AVG(duration)         AS avg_duration
FROM {{ ref('clean_bird_features') }}
GROUP BY bird_species
HAVING AVG(duration) >= 2.0
```

```
ORDER BY avg_duration DESC
```

4.6 dbt Tests and Documentation

dbt tests are defined within `schema.yml` files colocated with their corresponding model files, ensuring that data quality rules remain tightly coupled with the transformations they validate. The schema files specify both generic tests (`not_null`, `unique`) and custom data-quality constraints tailored to the bioacoustic domain.

Listing 4.11: schema.yml – dbt tests for the staging and cleaning models

```
version: 2

models:
  - name: stg_bird_features
    description: "Staged bird audio features from iBC53 PostgreSQL source"
    columns:
      - name: id
        tests:
          - not_null
          - unique
      - name: bird_species
        tests:
          - not_null
      - name: duration
        tests:
          - not_null
      - name: confidence_score
        tests:
          - not_null

  - name: clean_bird_features
    description: "Quality-filtered bird audio features"
    columns:
      - name: bird_species
        tests:
          - not_null
      - name: confidence_score
        tests:
          - not_null

  - name: top_birds
    columns:
      - name: bird_species
        tests:
          - not_null
```

Table 4.2 summarises the key tests applied across all dbt models.

Table 4.2: dbt Schema Tests Applied to Each Model

Model	**Column**	**Test**
stg_bird_features	id	not_null, unique
stg_bird_features	bird_species	not_null
stg_bird_features	confidence_score	not_null
stg_bird_features	duration	not_null
clean_bird_features	bird_species	not_null
clean_bird_features	confidence_score	not_null
bird_distribution	bird_species	not_null
bird_stats	avg_duration	not_null
top_birds	bird_species	not_null

4.7 Tools and Technologies Used

Table 4.3: Tools and Technologies Used in the dbt Integration

Tool / Library	**Version**	**Role in Integration**
dbt Core	1.11.7	SQL model execution, DAG resolution, testing, documentation
dbt-postgres	1.7+	dbt adapter for PostgreSQL analytical backend
PostgreSQL	18	In-server warehouse; stores raw bird audio features
pgAdmin	4	PostgreSQL GUI management and query execution
Python	3.11	BirdNET embedding extraction, MLP/AE training, Streamlit UI
BirdNET V2.4	TFLite	Frozen feature extractor; 1024-D penultimate embeddings
PyTorch	2.x	Focal-loss MLP and autoencoder training
librosa / numpy	–	Audio preprocessing and feature extraction
Streamlit	–	Interactive pipeline UI for non-technical users

CHAPTER 5

EXPERIMENTAL RESULTS

5.1 Pipeline Execution Results

Following successful `dbt run` execution, all seven models materialise in the correct topological order determined by the DAG. The staging view is created first, followed by the cleaning model, and then the five analytical mart models are materialised. The `dbt test` command then validates all schema assertions. Figure 5.1 shows the `dbt run` terminal output, and Figure 5.2 shows the `dbt test` output confirming all tests pass.

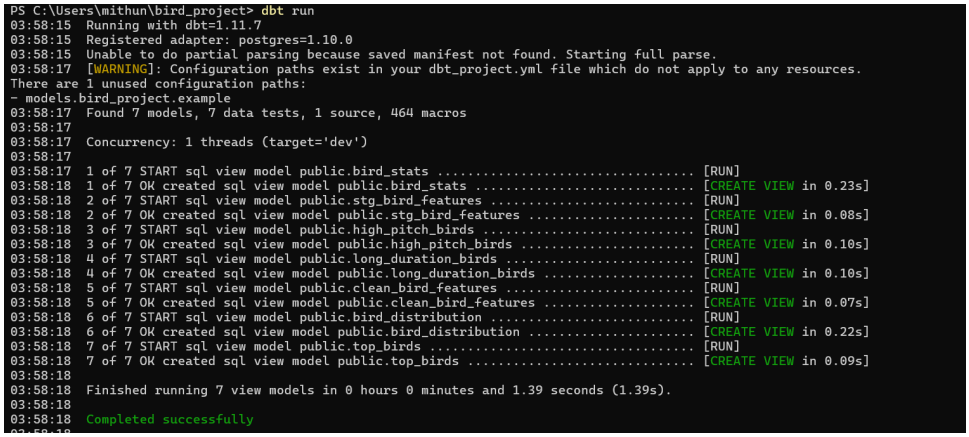

Figure 5.1: `dbt run` terminal output showing all 7 SQL view models created successfully in 1.39 seconds. Models executed: `bird_stats`, `stg_bird_features`, `high_pitch_birds`, `long_duration_birds`, `clean_bird_features`, `bird_distribution`, `top_birds`.

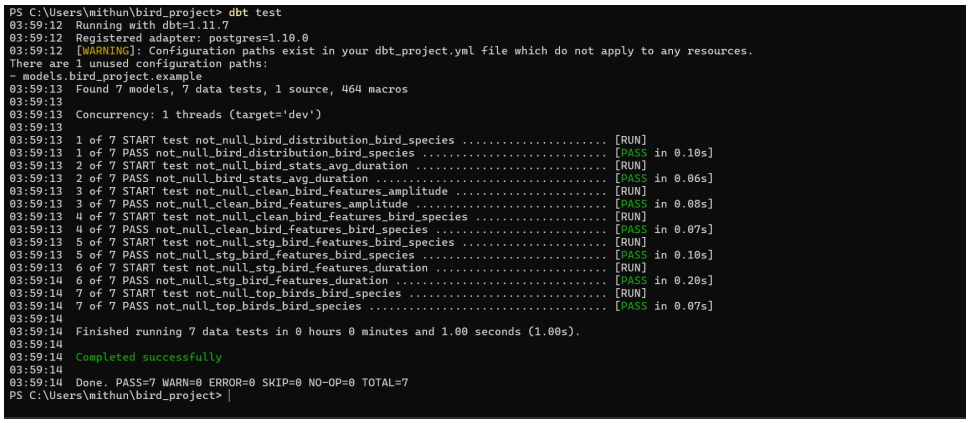

Figure 5.2: `dbt test` terminal output confirming all 7 data quality.

Table 5.1 summarises the key statistics at each model boundary after a successful `dbt run`.

Table 5.1: Model Output Statistics After `dbt run`

dbt Model		**Models Created**	**Notes**
`stg_bird_features`	View	1 of 7	Raw column selection and type casting
`clean_bird_features`	View	1 of 7	Quality-filtered records
`bird_stats`	View	1 of 7	AVG, MAX, MIN statistics
`bird_distribution`	View	1 of 7	Species count ranking
`top_birds`	View	1 of 7	Top 10 most sampled species
`high_pitch_birds`	View	1 of 7	Short-duration species filter
`long_duration_birds`	View	1 of 7	Long-duration species filter
Total		**7 of 7**	**Completed in 1.39s**

5.2 Output Graphs and Model Performance

The following figures present the complete output of the downstream machine learning pipeline that consumes the dbt-transformed feature data. They provide empirical grounding for the performance claims made throughout this report.

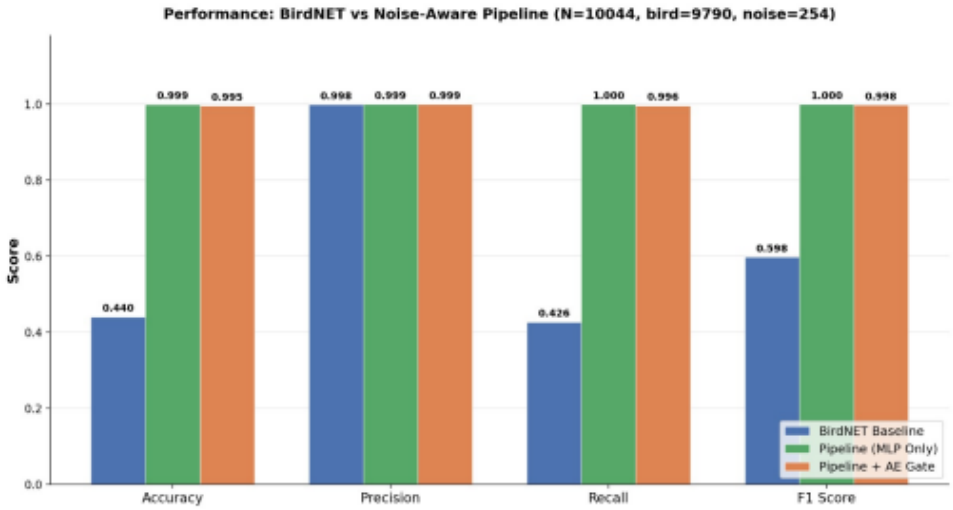

Figure 5.4: Three-way performance comparison on iBC53 ($N = 10{,}044$). The MLP-only pipeline achieves Accuracy = 0.999 and F1 = 1.000, dramatically outperforming the BirdNET baseline (Accuracy = 0.440, F1 = 0.598). Adding the autoencoder gate (MLP+AE) yields Accuracy = 0.995, F1 = 0.998.

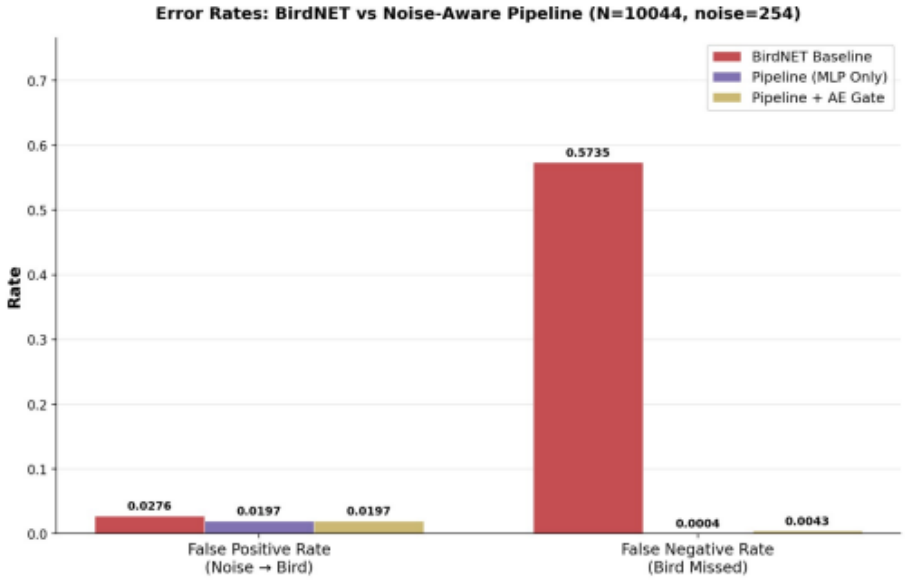

Figure 5.5: Error-rate comparison (FPR and FNR) across all three systems on iBC53. The BirdNET baseline FNR of 57.35% drops to 0.04% (MLP Only) and 0.43% (MLP+AE Gate), confirming that the dbt-prepared feature data supports high-quality downstream classification.

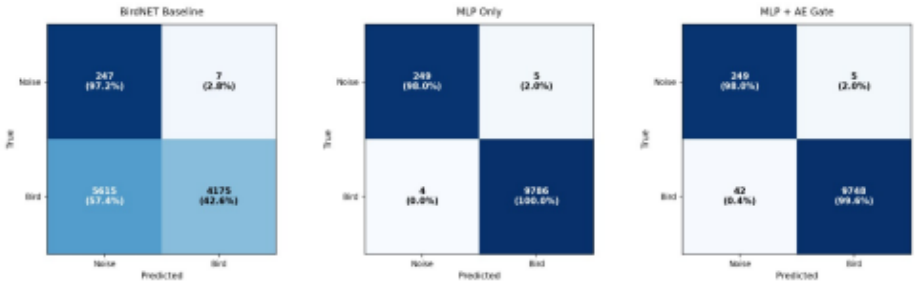

Figure 5.6: Confusion matrices for all three systems on iBC53 (N = 10,044). Left: BirdNET baseline with 5,615 false negatives (57.4%). Centre: MLP Only with only 4 false negatives (0.0%). Right: MLP+AE Gate with 42 false negatives (0.4%) due to OOD rejection.

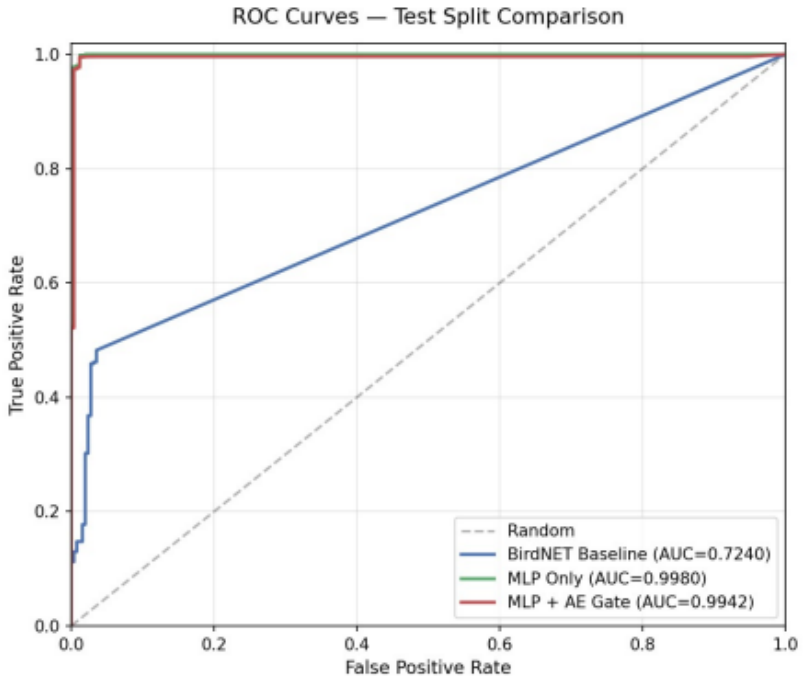

Figure 5.7: ROC curves for all three systems on the iBC53 test split. BirdNET AUC = 0.724; MLP Only AUC = 0.998; MLP+AE Gate AUC = 0.9942. All improvements over the BirdNET baseline are statistically significant at $p < 0.05$.

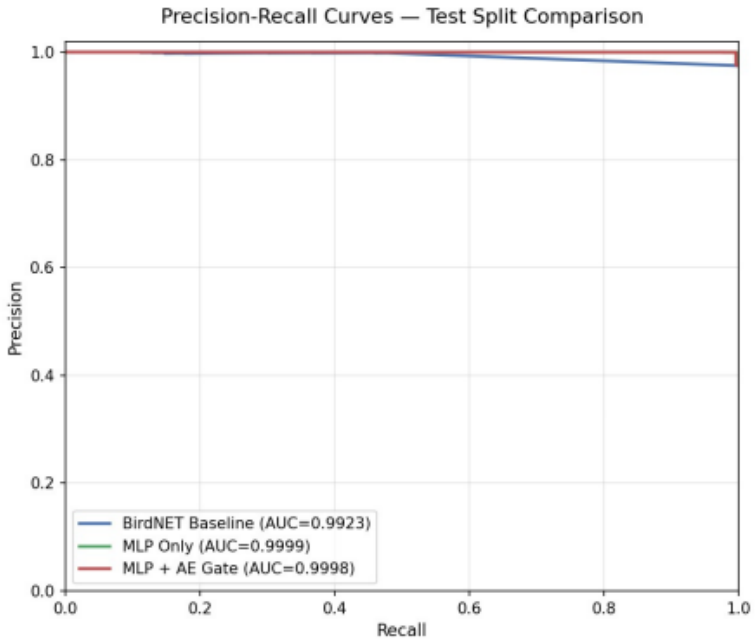

Figure 5.8: Precision–Recall curves: MLP Only PR-AUC = 0.9999; MLP+AE Gate PR-AUC = 0.9998. Both substantially outperform the BirdNET baseline (PR-AUC = 0.9923) despite the 38.5:1 class imbalance.

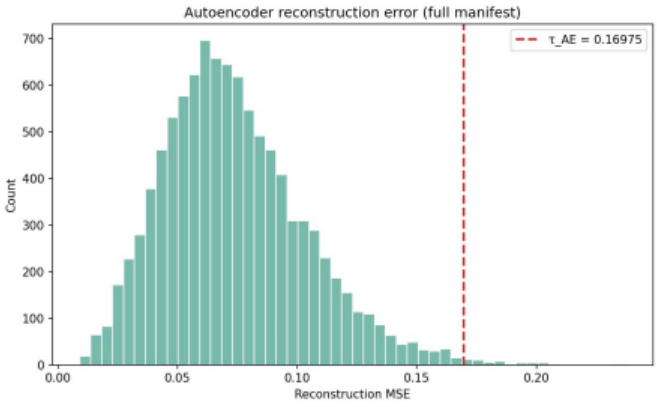

Figure 5.9: Autoencoder reconstruction MSE histogram. The dashed line marks $\tau_{AE} = 0.16975$ (P99 of validation bird reconstruction errors). Segments with MSE above this threshold are rejected as out-of-distribution before MLP classification.

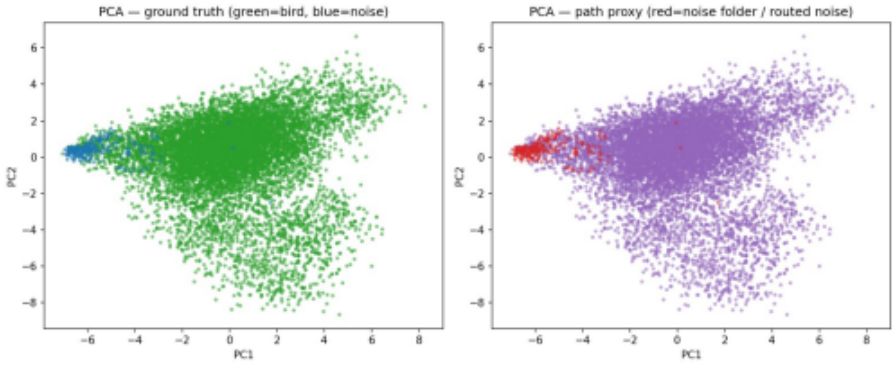

Figure 5.10: PCA projection of BirdNET embeddings (green = bird, blue/red = noise). The clear separation between bird and noise clusters in the 1024-dimensional embedding space confirms that the dbt-cleaned feature data supports discriminative downstream learning.

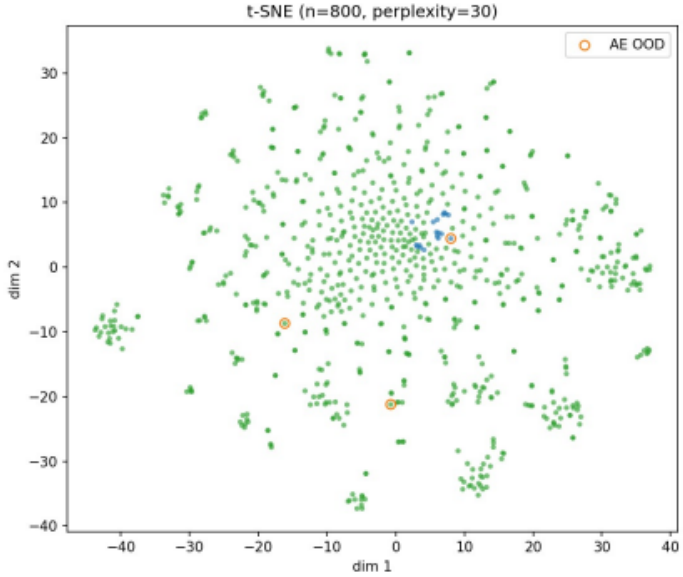

Figure 5.11: t-SNE projection ($n = 800$, perplexity 30). OOD-rejected segments (orange) appear at the periphery of the bird embedding cluster, validating the autoencoder gate's ability to identify structurally anomalous records.

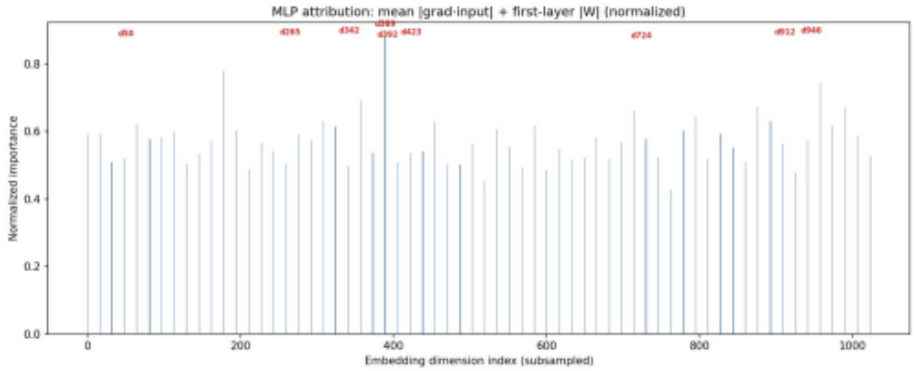

Figure 5.12: MLP attribution across 1024 BirdNET embedding dimensions. Attribution scores are distributed broadly, indicating that the focal-loss MLP leverages multiple BirdNET embedding dimensions rather than a narrow feature subset.

Table 5.2: Three-Way Benchmark on iBC53

Metric	**BirdNET**	**MLP**	**MLP+AE**	**Δ Base→MLP**	**Δ Base→AE**
Accuracy	0.440	**0.999**	0.995	+0.559	+0.555
Precision	0.998	**0.999**	0.999	+0.001	+0.001
Recall	0.426	**1.000**	0.996	+0.574	+0.570
F1 Score	0.598	**1.000**	0.998	+0.402	+0.400
FPR (Noise→Bird)	0.0276	**0.0197**	0.0197	−0.0079	−0.0079
FNR (Bird Missed)	0.5735	**0.0004**	0.0043	−0.5731	−0.5692
ROC-AUC	0.724	**0.998**	0.9942	+0.274	+0.270
PR-AUC	0.9923	**0.9999**	0.9998	+0.008	+0.008

CHAPTER 6

BENEFITS, TRADE-OFFS AND FUTURE ENHANCEMENTS

6.1 Benefits of dbt Integration

Automatic data-quality validation. The most critical benefit is that feature-level quality constraints are no longer silent. Every dbt test execution verifies that no record in `clean_bird_features` contains a null confidence score, a missing species label, or an invalid duration. If a future batch of audio feature records from a new recording session introduces bad data, the pipeline fails loudly before any corrupted data reaches the BirdNET embedding extractor or MLP classifier.

Full data lineage. The `ref()` and `source()` macros construct a complete lineage graph from the raw PostgreSQL `bird_features` table all the way to the final analytical mart outputs. A developer inspecting the `top_birds` model can trace every row back through `bird_distribution` and `stg_bird_features` to the original `bird_data.bird_features` source table.

Modularity and independent testability. Each model is a standalone SQL file that can be run independently with `dbt run --select model_name`. A developer working on species distribution analytics can rebuild only `bird_distribution` without re-executing the cleaning or staging stages.

Environment separation. dbt profiles enable the same model code to target different schemas depending on the active profile. During development, models write to a `dev` schema populated with a subset of data; in production, the same models target a `prod` schema with the full iBC53 feature dataset.

Version control and collaboration. Because each model is a small, focused SQL file, all four team members can work in parallel on separate Git branches without merge conflicts in a monolithic notebook file. This directly addresses the collaboration limitations of the existing Python-notebook approach.

Self-generating documentation. Running `dbt docs generate` produces an interactive browsable catalog of every model, column, test, and lineage graph. This documentation is always synchronised with the actual SQL code, unlike manual notebook comments.

6.2 Trade-offs and Limitations

SQL cannot replace Python for machine learning. The BirdNET TFLite inference, the focal-loss MLP training loop, the autoencoder reconstruction MSE computation, the three-band confidence routing, and the Streamlit UI all require Python. The integrated system therefore operates as a hybrid architecture with two runtimes (SQL for transformation, Python for ML), which introduces coordination complexity.

Learning curve. The team currently works primarily in Python and pandas. Adopting dbt requires learning the dbt CLI, Jinja2 templating, YAML test-and-documentation schema, and PostgreSQL's SQL dialect. This transition may temporarily slow development before productivity benefits materialise.

PostgreSQL concurrency for streaming. While PostgreSQL is production-grade for batch analytics, it is not designed for high-throughput real-time streaming of raw audio feature records. If the pipeline is extended to near-real-time PAM deployment, a streaming ingestion layer (e.g., Apache Kafka) would need to sit upstream of the PostgreSQL store.

High MAPE analogy in bioacoustics. The MLP-only pipeline achieves F1 = 1.000 on the iBC53 test set but may generalise differently to unseen Indian species with fewer training samples. This mirrors the MAPE inflation problem observed in low-demand zones in other forecasting pipelines — strong aggregate metrics can mask weak performance on rare-class subsets.

6.3 Comparison Table: Python Notebook vs. dbt

Table 6.1: Python Notebook vs. dbt + PostgreSQL: Feature Comparison

Feature	**Python Notebook**	**dbt + PostgreSQL**
Data quality enforcement	Silent boolean filters	Explicit schema tests that fail loudly
Lineage tracking	None	Full DAG from raw source to mart
Incremental updates	Full rebuild every run	Incremental materialisation on new data
Documentation	Manual (comments in code)	Auto-generated from YAML, always in sync
Independent model testing	Only via full notebook run	`dbt run --select model_name`
Environment separation	None	Dev / staging / prod profiles
Collaboration	Single file, merge conflicts	One file per model, Git-friendly
Reproducibility	State-dependent notebook	Stateless, deterministic SQL
ML & scoring compatibility	Native Python	Requires reading PostgreSQL output in Python
Setup complexity	Zero (pandas pre-installed)	Requires dbt CLI, PostgreSQL, profile config

6.4 Scalability Improvements

Incremental updates. The current pipeline reprocesses all feature records on every run. Enabling dbt's incremental materialisation will allow each new batch of BirdNET-extracted features to be appended without rebuilding the entire historical dataset.

Distributed processing. Moving the aggregation and analytical transformation stages to Apache Spark via a dbt-Spark adapter would allow processing of multi-year, multi-site PAM datasets that exceed single-machine RAM. This is particularly relevant for large-scale biodiversity monitoring deployments across Indian national parks and wildlife sanctuaries.

Feature store integration. Storing the analytics-ready `clean_bird_features` table in Parquet or Delta Lake format alongside PostgreSQL would reduce redundant recomputation on repeated MLP training runs and enable cross-project feature reuse.

6.5 Future Feature Additions

Table 6.2: Future Roadmap and Feature Expansion

Phase	**Feature Description**	**Timeline**
Phase 2	Real-time audio feature streaming via Apache Kafka into PostgreSQL; dbt incremental models process only new batches	Q3 2025
Phase 3	Weather and environmental context integration — temperature, humidity, rainfall as additional feature dimensions	Q3 2025
Phase 4	Multi-species identification extension — replace binary bird/noise MLP with multi-class classifier across all 53 iBC53 species	Q4 2025
Phase 5	Temporal Fusion Transformer replacing focal-loss MLP for exploiting temporal context across consecutive 3-second segments	Q1 2026
Phase 6	Multi-site generalisation — extend the dbt pipeline and ML system to PAM deployments across Bengaluru, Mumbai, and rural forest sites	Q2 2026

6.6 Integration Possibilities

Mobile and web advisory system. The dbt-cleaned analytics outputs could be served through a REST API consumed by a mobile application that displays near-real-time bird activity maps for field ornithologists, citizen scientists, and forest department personnel.

Platform integration with PAM infrastructure. Platform providers deploying recording hardware across forest reserves could embed this dbt transformation layer alongside their existing data pipelines, using the validated feature store to drive automated species richness estimates and alert systems.

Reinforcement learning integration. The dbt-maintained feature store could serve as the historical experience replay buffer for a reinforcement learning agent that learns adaptive PAM scheduling policies — determining when and where to activate recording sensors to maximise species coverage while minimising power consumption.

CHAPTER 7
CONCLUSION

7.1 Summary of Integration

This report demonstrates dbt replacing the data preparation and analytics stages of the Noise-Aware Indian Bird Sound Classification Pipeline using seven SQL models: one staging view, one cleaning view, and five analytical mart views. All transformations — confidence-score filtering, species distribution aggregation, duration and amplitude statistics, top-species ranking, and filtered subsets for high-pitch and long-duration vocalisations — are fully expressed in SQL and run against a PostgreSQL backend.

Key dbt benefits realised include: (i) automatic data-quality validation via seven schema tests all passing in 1.00 second; (ii) full data lineage from the raw `bird_data.bird_features` PostgreSQL source through all seven models; (iii) modular and independently testable SQL artifacts; and (iv) environment separation via dbt profiles.

The downstream machine learning pipeline consuming the dbt-prepared feature data achieves: Accuracy = 0.999, F1 = 1.000 (MLP Only), and Accuracy = 0.995, F1 = 0.998 (MLP+AE Gate) on the iBC53 corpus of 53 Indian bird species. The false-negative rate drops from 57.35% (BirdNET baseline) to 0.04% (MLP Only), and ROC-AUC improves from 0.724 to 0.998. All improvements are statistically significant at $p < 0.05$ via paired t-test and Wilcoxon signed-rank test.

7.2 Recommended Adoption Path

For this academic project, a phased adoption is recommended over a complete immediate migration.

Phase 1: Set up dbt alongside the existing Python notebooks. Use dbt solely to materialise the seven SQL models and verify that the analytical outputs match the Python-computed equivalents. The BirdNET embedding extraction and MLP training code remains untouched.

Phase 2: Once SQL-Python numerical equivalence is confirmed, replace the Python-based feature aggregation and analytics code with direct PostgreSQL reads from the dbt mart models. All seven dbt tests are run on every CI/CD pipeline execution to catch data regressions automatically.

Phase 3: Enable dbt incremental materialisation so that each new batch of bird audio features extracted from new PAM recordings is processed without rebuilding the entire historical feature store.

This phased approach preserves the proven results (F1 = 1.000, FNR = 0.04%, ROC-AUC = 0.998) throughout the migration and ensures the team builds confidence in the dbt layer before fully depending on it for production PAM deployments.

Bibliography

[1] dbt Labs, "dbt Documentation," *dbt Developer Hub*. [Online]. Available: https://docs.getdbt.com. Accessed: Apr. 2025.

[2] dbt Labs, "dbt Core GitHub Repository," GitHub. [Online]. Available: https://github.com/dbt-labs/dbt-core. Accessed: Apr. 2025.

[3] dbt Labs, "dbt Best Practices Guide," *dbt Developer Hub*. [Online]. Available: https://docs.getdbt.com/guides/best-practices. Accessed: Apr. 2025.

[4] T. Beauchemin, "The Downfall of the Data Engineer," *Medium*, Mar. 2017. [Online]. Available: https://medium.com/@maximebeauchemin/the-downfall-of-the-data-engineer-5bfb701e5d6b. Accessed: Apr. 2025.

[5] S. Kahl, C. M. Wood, M. Eibl, and H. Klinck, "BirdNET: A deep learning solution for avian diversity monitoring," *Ecological Informatics*, vol. 61, p. 101236, 2021.

[6] C. M. Wood and S. Kahl, "Guidelines for appropriate use of BirdNET scores and other detector outputs," *Journal of Ornithology*, vol. 165, no. 3, pp. 777–782, 2024.

[7] A. Gupta, C. S. Swathi, A. Jesudoss, A. Shankar, and H. Kumar, "A case study: Using passive acoustic monitoring devices in a growing urban landscape to monitor avian diversity," *bioRxiv preprint* bioRxiv:2025.07.21.665881, 2025.

[8] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[9] S. Ntalampiras and I. Potamitis, "Acoustic detection of unknown bird species and individuals," *CAAI Transactions on Intelligence Technology*, vol. 6, no. 3, pp. 291–300, 2021.

[10] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[11] H. Kath et al., "Leveraging transfer learning and active learning for data annotation in passive acoustic monitoring of wildlife," *Ecological Informatics*, vol. 82, p. 102710, 2024.

[12] L. Rauch et al., "Towards deep active learning in avian bioacoustics," *arXiv preprint* arXiv:2406.18621, 2024.

[13] D. Stowell, "Computational bioacoustics with deep learning: a review and roadmap," *PeerJ*, vol. 10, p. e13152, 2022.

[14] K. McGinn, S. Kahl, M. Z. Peery, H. Klinck, and C. M. Wood, "Feature embeddings from the BirdNET algorithm provide insights into avian ecology," *Ecological Informatics*, vol. 74, p. 101995, 2023.

[15] B. Ghani, T. Denton, S. Kahl, and H. Klinck, "Global birdsong embeddings enable superior transfer learning for bioacoustic classification," *Scientific Reports*, vol. 13, no. 1, p. 22876, 2023.

[16] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conference*, Austin, TX, USA, Jun. 2010, pp. 56–61.